# Module 7 Lab: Session 3: Long Work Without Blocking the Request

| | |
|---|---|
| **Module** | M7: Advanced Web API Development |
| **Session** | 3 of 4 |
| **Exercises** | 5 (Background workers + status store + idempotency), 6 (SignalR groups + reconnect) |

## Welcome to "Don't Block the Request Thread"

The registrar opens the new V2 admin dashboard and clicks **Generate Transcript** for student #4271. The page locks. Five seconds. Ten. Fifteen. The browser's "this page is unresponsive" dialog appears. She clicks **Wait** out of habit. At second 22 the response arrives  a 200 OK with a PDF link.

She clicks **Generate Transcript** again, this time for student #4272. Same lock. Same wait. By the end of the morning she has spent thirty minutes staring at a beachball.

Worse: a colleague double-clicked the button at second 14 by mistake. Two transcripts were generated. Two PDFs went out by email. The student got two duplicate transcripts and called support.

**Both problems share a root cause:** the API is doing the work *during* the HTTP request. The fix is the **Async Request-Reply** pattern (Exercise 5);POST returns 202 instantly with a status URL, and a `BackgroundService` does the actual work. The duplicate fix is **idempotency keys**;the second click with the same key returns the original 202 with the original report id, no second worker job.

Then there is the polling problem. A 202 returns a status URL;the dashboard now has to poll it every two seconds for fifteen. That works, and it is the safe default, but for users on the page right now the right answer is to **push** the "your transcript is ready" notification the moment it happens. That is Exercise 6, with SignalR's typed hubs and group dispatch.

**Session 3 builds the production pattern for any work that takes longer than a request can wait.** Reports, exports, certificate generation, video transcoding; all the same shape.

## Before You Begin: Session 3 sync

Confirm your Session 1 + 2 work still passes:

```
dotnet build
```

**Primary checks (Scalar):** Open **Try it** for **`GET /api/v2/courses`** (cache + rate-limiter path). POST **`/api/v2/enrollments`** with body `{"studentId":1,"courseCode":"CSE-101"}` (or a valid student/course in your seed);confirm **202/201** behaviour matches your Session 1 CQRS design. Reach Scalar from the same HTTPS origin **`dotnet run`** prints; try **`/scalar/v1`**, then **`/scalar`** if needed.

Optional scripted replay:

```
curl https://localhost:5001/api/v2/courses
curl -i -X POST https://localhost:5001/api/v2/enrollments \
  -H "Content-Type: application/json" \
  -d '{"studentId":1,"courseCode":"CSE-101"}'
```

If anything fails, fix it before continuing,  Session 3's transcript controller must sit on the same rate-limit and exception-handling plumbing.

Install the Session 3 package:

```
dotnet add TmsApi.Api package Microsoft.AspNetCore.SignalR
```

## Exercise 5: Background Workers, Status Store, and Idempotency (LO 7.4)

When an admin requests a student transcript, the generation takes 5–15 seconds. The current code returns 202 Accepted but **`GET /api/v2/transcripts/{id}/status`** always says `"Processing"` even after the worker logged "complete." That is a teaching bug, not a feature. You will fix it: a real status store with a proper state machine, an idempotency-key check so a double-clicked POST does not enqueue twice, and a status endpoint that reflects truth. This is the version that survives an interview.

**Session 2 → 5 bridge:** If you added the **Exercise 4** minimal `POST /api/v2/transcripts` stub (`200 OK` + `[EnableRateLimiting("transcripts")]`), **replace that action** with the full `TranscriptsController` below,  keep **`[EnableRateLimiting("transcripts")]`** on the POST so concurrency limits from Session 2 still apply.

## Step 1 Define the contract: request, state, and status

Create `TmsApi.Application/Transcripts/TranscriptModels.cs`:

```
namespace TmsApi.Application.Transcripts;

public enum TranscriptState { Queued, Processing, Ready, Failed }

public record TranscriptRequest(int StudentId, string? ReportId = null)
{
    public TranscriptRequest WithReportId(string id) => this with { Rep
ortId = id };
}

public record TranscriptStatus(
    string ReportId,
    int StudentId,
    TranscriptState State,
    DateTimeOffset RequestedAt,
    DateTimeOffset? StartedAt = null,
    DateTimeOffset? CompletedAt = null,
    string? DownloadUrl = null,
    string? ErrorMessage = null);
```

> The state machine is `Queued → Processing → (Ready | Failed)`. There is no path from `Failed` back to `Queued` a failed transcript requires a new POST with a fresh report id. That keeps the status endpoint honest: a 200 OK transcript stays 200 OK, not "we'll try again silently."

## Step 2 Build a status store with a state machine

In-memory for the lab; the same interface points at Redis/SQL/DynamoDB for production. Create `TmsApi.Infrastructure/Transcripts/ITranscriptStatusStore.cs`:

```
using TmsApi.Application.Transcripts;

namespace TmsApi.Infrastructure.Transcripts;

public interface ITranscriptStatusStore
{
    Task<TranscriptStatus> CreateAsync(string reportId, int studentId,
CancellationToken ct);
    Task MarkProcessingAsync(string reportId, CancellationToken ct);
    Task MarkReadyAsync(string reportId, string downloadUrl, Cancellati
onToken ct);
    Task MarkFailedAsync(string reportId, string error, CancellationTok
en ct);
```

```
    Task<TranscriptStatus?> GetAsync(string reportId, CancellationToken
 ct);

    // Idempotency
    Task<string?> GetReportIdForIdempotencyKeyAsync(string idempotencyK
ey, CancellationToken ct);
    Task LinkIdempotencyKeyAsync(string idempotencyKey, string reportId,
 CancellationToken ct);
}
```

Create the in-memory implementation

`TmsApi.Infrastructure/Transcripts/InMemoryTranscriptStatusStore.cs`:

```
using System.Collections.Concurrent;
using TmsApi.Application.Transcripts;

namespace TmsApi.Infrastructure.Transcripts;

public class InMemoryTranscriptStatusStore : ITranscriptStatusStore
{
    private readonly ConcurrentDictionary<string, TranscriptStatus> _by
ReportId = new();
    private readonly ConcurrentDictionary<string, string> _idempotencyT
oReportId = new();

    public Task<TranscriptStatus> CreateAsync(string reportId, int stud
entId, CancellationToken ct)
    {
        var status = new TranscriptStatus(
            reportId, studentId, TranscriptState.Queued,
            RequestedAt: DateTimeOffset.UtcNow);
        _byReportId[reportId] = status;
        return Task.FromResult(status);
    }

    public Task MarkProcessingAsync(string reportId, CancellationToken
ct) =>
        Transition(reportId, current => current with
        {
            State = TranscriptState.Processing,
            StartedAt = DateTimeOffset.UtcNow
        }, allowedFrom: TranscriptState.Queued);

    public Task MarkReadyAsync(string reportId, string downloadUrl, Can
cellationToken ct) =>
        Transition(reportId, current => current with
        {
            State = TranscriptState.Ready,
```

```
            CompletedAt = DateTimeOffset.UtcNow,
            DownloadUrl = downloadUrl
        }, allowedFrom: TranscriptState.Processing);

    public Task MarkFailedAsync(string reportId, string error, Cancella
tionToken ct) =>
        Transition(reportId, current => current with
        {
            State = TranscriptState.Failed,
            CompletedAt = DateTimeOffset.UtcNow,
            ErrorMessage = error
        }, allowedFrom: TranscriptState.Processing);

    public Task<TranscriptStatus?> GetAsync(string reportId, Cancellati
onToken ct) =>
        Task.FromResult(_byReportId.TryGetValue(reportId, out var s) ?
s : null);

    public Task<string?> GetReportIdForIdempotencyKeyAsync(string key,
CancellationToken ct) =>
        Task.FromResult(_idempotencyToReportId.TryGetValue(key, out var
 id) ? id : null);

    public Task LinkIdempotencyKeyAsync(string key, string reportId, Ca
ncellationToken ct)
    {
        _idempotencyToReportId.TryAdd(key, reportId);
        return Task.CompletedTask;
    }

    private Task Transition(string reportId, Func<TranscriptStatus, Tra
nscriptStatus> change, TranscriptState allowedFrom)
    {
        if (!_byReportId.TryGetValue(reportId, out var current))
            throw new InvalidOperationException($"Unknown report id {re
portId}.");

        if (current.State != allowedFrom)
            throw new InvalidOperationException(
                $"Cannot move {reportId} from {current.State} via this
transition (expected {allowedFrom}).");

        _byReportId[reportId] = change(current);
        return Task.CompletedTask;
    }
}
```

A `ConcurrentDictionary` is enough for a single-process lab. Production swaps the implementation for Redis (TTL + atomic ops) or a SQL `transcript_status` table with an index on `report_id`. The interface stays identical the controller and worker do not care.

Register it as a singleton in `Program.cs`:

```
builder.Services.AddSingleton<ITranscriptStatusStore, InMemoryTranscrip
tStatusStore>();
```

## Step 3 Create the bounded channel

Open `Program.cs`:

```
using System.Threading.Channels;

builder.Services.AddSingleton(Channel.CreateBounded<TranscriptRequest>(
    new BoundedChannelOptions(100)
    {
        FullMode = BoundedChannelFullMode.Wait
    }));
```

## Step 4 Create the background worker that updates the store

Because **`TmsApi.Infrastructure`** is a class library project (**`classlib`**), it does not implicitly reference ASP.NET Core framework abstractions like **`BackgroundService`**. Add **`Microsoft.Extensions.Hosting.Abstractions`** to the Infrastructure project before creating the worker:

```
dotnet add TmsApi.Infrastructure package Microsoft.Extensions.Hosting.Abstractions
```

Create `TmsApi.Infrastructure/Workers/TranscriptWorker.cs`:

```
using System.Threading.Channels;
using TmsApi.Application.Transcripts;
using TmsApi.Infrastructure.Transcripts;

namespace TmsApi.Infrastructure.Workers;

public class TranscriptWorker(
    Channel<TranscriptRequest> channel,
    IServiceScopeFactory scopeFactory,
    ITranscriptStatusStore statusStore,
    ILogger<TranscriptWorker> logger)
    : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        logger.LogInformation("Transcript worker started.");
```

```
        await foreach (var request in channel.Reader.ReadAllAsync(ct))
        {
            var reportId = request.ReportId
                ?? throw new InvalidOperationException("ReportId must b
e set before queueing.");

            try
            {
                await statusStore.MarkProcessingAsync(reportId, ct);
                logger.LogInformation(
                    "Generating transcript {ReportId} for student {Stud
entId}",
                    reportId, request.StudentId);

                using var scope = scopeFactory.CreateScope();
                // Real production: pull the EF context, render PDF, sa
ve to blob storage.
                await Task.Delay(TimeSpan.FromSeconds(5), ct);

                var downloadUrl = $"/api/v2/transcripts/{reportId}/down
load";
                await statusStore.MarkReadyAsync(reportId, downloadUrl,
 ct);

                logger.LogInformation("Transcript ready: {ReportId}", r
eportId);
            }
            catch (OperationCanceledException) when (ct.IsCancellationR
equested)
            {
                logger.LogWarning("Worker shutdown  transcript {ReportI
d} did not complete", reportId);
                throw;
            }
            catch (Exception ex)
            {
                logger.LogError(ex, "Failed to generate transcript {Rep
ortId}", reportId);
                await statusStore.MarkFailedAsync(reportId, ex.Message,
 CancellationToken.None);
            }
        }
    }
}
```

Register:

```
builder.Services.AddHostedService<TranscriptWorker>();
```

## Step 5 Controller with idempotency-key handling

Create `TmsApi.Api/Controllers/V2/TranscriptsController.cs`:

```
using System.Threading.Channels;
using Asp.Versioning;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.RateLimiting;
using TmsApi.Application.Transcripts;
using TmsApi.Infrastructure.Transcripts;

namespace TmsApi.Api.Controllers.V2;

[ApiController]
[Route("api/v2/transcripts")]
[ApiVersion("2.0")]
public class TranscriptsController(
    Channel<TranscriptRequest> channel,
    ITranscriptStatusStore statusStore) : ControllerBase
{
    [HttpPost]
    [EnableRateLimiting("transcripts")]
    public async Task<IActionResult> RequestTranscript(
        TranscriptRequest request,
        [FromHeader(Name = "Idempotency-Key")] string? idempotencyKey,
        CancellationToken ct)
    {
        if (!string.IsNullOrWhiteSpace(idempotencyKey))
        {
            var existing = await statusStore.GetReportIdForIdempotencyK
eyAsync(idempotencyKey, ct);
            if (existing is not null)
            {
                var existingStatus = await statusStore.GetAsync(existin
g, ct);
                return Accepted(
                    Url.Action(nameof(GetStatus), new { id = existing
 }),
                    existingStatus);
            }
        }

        var reportId = Guid.NewGuid().ToString("N")[..12];
        var status = await statusStore.CreateAsync(reportId, request.St
udentId, ct);

        if (!string.IsNullOrWhiteSpace(idempotencyKey))
            await statusStore.LinkIdempotencyKeyAsync(idempotencyKey, r
eportId, ct);
```

```
        await channel.Writer.WriteAsync(request.WithReportId(reportId),
 ct);

        Response.Headers.RetryAfter = "5";
        return Accepted(
            Url.Action(nameof(GetStatus), new { id = reportId }),
            status);
    }

    [HttpGet("{id}/status")]
    public async Task<IActionResult> GetStatus(string id, CancellationT
oken ct)
    {
        var status = await statusStore.GetAsync(id, ct);
        return status is null
            ? NotFound(new ProblemDetails
            {
                Title = "Transcript not found",
                Detail = $"No transcript request with id '{id}'.",
                Status = StatusCodes.Status404NotFound
            })
            : Ok(status);
    }
}
```

The `Idempotency-Key` header is the same pattern Stripe popularised: the *client* picks a unique value (a UUID is the convention) and the server promises that two POSTs with the same key produce the same logical effect. A double-clicked enrollment, a retry after a flaky network, or a duplicate webhook all become safe the second call returns the original 202 with the original report id, no second worker job. The `Retry-After: 5` header gives the client a polite hint about polling cadence (transcripts take ~5 seconds in this lab).

## Step 6 Run the full flow

```
dotnet run --project TmsApi.Api
```

Issue the first request with an idempotency key:

```
curl -i -X POST https://localhost:5001/api/v2/transcripts \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: 11111111-2222-3333-4444-555555555555" \
  -d '{"studentId": 1}'
```

Repeat the **exact same request** within a couple of seconds. You should receive 202 with the **same** `reportId` and a status of `Queued` or `Processing` no new worker job kicked off. Then poll:

```
curl https://localhost:5001/api/v2/transcripts/<reportId>/status
```

**Expected result over time:**

| Time after POST | `state` | `startedAt` | `completedAt` | `downloadUrl` |
|---|---|---|---|---|
| 0 s | `Queued` | null | null | null |
| ~0.1 s | `Processing` | set | null | null |
| ~5 s | `Ready` | set | set | `/api/v2/.../download` |

A second POST with the same `Idempotency-Key`:

- Returns 202 with the **same** `reportId`.
- Does *not* trigger a second worker run (logs show only one "Generating transcript …" line for that id).

Querying a non-existent id returns `404` with `ProblemDetails`.

## Step 7 A note on durability

`InMemoryTranscriptStatusStore` is single-process. If the API restarts, in-flight transcripts and idempotency keys vanish. Production options, in increasing weight:

1. **Redis** with TTL on the idempotency-key entries (24h is the conventional Stripe default).
2. **A SQL `transcript_status` table** read by the worker on startup survives restarts, supports multiple workers behind a leader-election lock.
3. **A real broker** Azure Service Bus, RabbitMQ, AWS SQS when the worker needs to scale to a separate process. The Channel-based pattern in this lab maps 1:1 to a broker queue; the controller's `WriteAsync` becomes a `SendMessageAsync` and nothing else changes.

You do not need to swap implementations in this lab. You do need to be able to explain, in interview, which option you would choose for which scale.

## Checkpoint

`POST /api/v2/transcripts` returns 202 immediately. The status endpoint reflects truth `Queued`, then `Processing`, then `Ready` with a real `downloadUrl` and

timestamps. A double POST with the same `Idempotency-Key` is safely deduplicated. The state machine refuses illegal transitions. Failures are recorded as `Failed` with an error message rather than disappearing into logs. The pattern lifts cleanly to Redis or a message broker without changing the controller.

## Exercise 6: SignalR with Groups, Reconnect, and Backplane Awareness (LO 7.5)

When a student's transcript is ready, their dashboard should update without polling. The naive answer "broadcast to everyone, the client filters" leaks information and burns bandwidth. The interview-correct answer uses **groups**, the **strongly-typed hub**, and **automatic reconnection** on the client. Authorization will land in M12; until then, students join their own group on connect using the student id they pass on the query string. You will build that, plus understand what you would change to scale to two pods.

### Step 1 Define the hub client interface

Create `TmsApi.Application/Hubs/ITmsHubClient.cs`:

```
namespace TmsApi.Application.Hubs;

public interface ITmsHubClient
{
    Task ReceiveTranscriptReady(string reportId, string downloadUrl);
    Task ReceiveCourseUpdate(string courseCode, string message);
    Task ReceiveGradePosted(string courseCode, int studentId, decimal g
rade);
}
```

### Step 2 Create the hub with student auto-join

Create `TmsApi.Api/Hubs/TmsHub.cs`. The hub auto-joins the student to their own group on connect (using `studentId` from the connection query string), and exposes explicit join/leave for course-level groups:

```
using Microsoft.AspNetCore.SignalR;
using TmsApi.Application.Hubs;

namespace TmsApi.Api.Hubs;

public class TmsHub : Hub<ITmsHubClient>
{
    public override async Task OnConnectedAsync()
```

```
    {
        var studentId = Context.GetHttpContext()?.Request.Query["studen
tId"].ToString();
        if (!string.IsNullOrWhiteSpace(studentId))
        {
            await Groups.AddToGroupAsync(Context.ConnectionId, GroupNam
es.Student(studentId));
        }
        await base.OnConnectedAsync();
    }

    public async Task JoinCourseGroup(string courseCode)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, GroupNames.C
ourse(courseCode));
    }

    public async Task LeaveCourseGroup(string courseCode)
    {
        await Groups.RemoveFromGroupAsync(Context.ConnectionId, GroupNa
mes.Course(courseCode));
    }

    public override async Task OnDisconnectedAsync(Exception? exception)
    {
        // SignalR removes the connection from all groups automatically.
        await base.OnDisconnectedAsync(exception);
    }
}

public static class GroupNames
{
    public static string Student(string studentId) => $"student-{studen
tId}";
    public static string Course(string courseCode) => $"course-{courseC
ode}";
}
```

> Why a query-string `studentId` rather than `Clients.User(...)`? `Clients.User` requires SignalR to identify the user via `IUserIdProvider`, which in practice means JWT auth which the curriculum lands in **M12**. Until then, a per-student group is functionally identical for this lab and gives learners the production-correct mental model: **never broadcast to all, always send to a group**. The day M12 wires JWT, swap auto-join from `studentId` query to `Context.UserIdentifier` and the rest of the code does not change.

> Centralising group names in a static class (`GroupNames.Student(...)`) is the production discipline. The day someone typos `student-` as `studnet-`, this lab catches it in code review instead of in production silence.

## Step 3 Register SignalR

In `Program.cs`:

```
builder.Services.AddSignalR();

// After app.Build()
app.MapHub<TmsHub>("/hubs/tms");
```

## Step 4 Create the notification abstraction

`TranscriptWorker` lives in the Infrastructure layer. `TmsHub` lives in the Api layer. In Clean Architecture, Infrastructure cannot reference Api — dependencies point inward. The fix is a notification interface in Application that the worker calls, with the SignalR implementation living in Api where `TmsHub` is visible. The worker never learns that SignalR exists.

Create `TmsApi.Application/Notifications/ITranscriptNotificationService.cs`:

```
namespace TmsApi.Application.Notifications;

public interface ITranscriptNotificationService
{
    Task NotifyTranscriptReadyAsync(int studentId, string reportId, str
ing downloadUrl);
}
```

Create `TmsApi.Api/Notifications/SignalRTranscriptNotificationService.cs`:

```
using Microsoft.AspNetCore.SignalR;
using TmsApi.Api.Hubs;
using TmsApi.Application.Hubs;
using TmsApi.Application.Notifications;

namespace TmsApi.Api.Notifications;

public class SignalRTranscriptNotificationService(IHubContext<TmsHub, I
TmsHubClient> hubContext)
    : ITranscriptNotificationService
{
    public async Task NotifyTranscriptReadyAsync(int studentId, string
```

```
reportId, string downloadUrl)
    {
        await hubContext.Clients
            .Group(GroupNames.Student(studentId.ToString()))
            .ReceiveTranscriptReady(reportId, downloadUrl);
    }
}
```

Register the notification service in `Program.cs` (alongside the `AddSignalR` call from Step 3):

```
builder.Services.AddSingleton<ITranscriptNotificationService, SignalRTr
anscriptNotificationService>();
```

> Why `AddSingleton`? `IHubContext<TmsHub, ITmsHubClient>` is itself a singleton, the DI container hands out the same hub context for the lifetime of the app. The notification service holds no mutable state, so singleton is the natural lifetime.

## Step 5 Update the worker to notify through the abstraction

Update `TranscriptWorker` to inject `ITranscriptNotificationService` instead of `IHubContext`. Replace your worker class from Exercise 5:

```
public class TranscriptWorker(
    Channel<TranscriptRequest> channel,
    ITranscriptStatusStore statusStore,
    ITranscriptNotificationService notificationService,
    ILogger<TranscriptWorker> logger)
    : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        logger.LogInformation("Transcript worker started.");

        await foreach (var request in channel.Reader.ReadAllAsync(ct))
        {
            var reportId = request.ReportId!;
            try
            {
                await statusStore.MarkProcessingAsync(reportId, ct);
                await Task.Delay(TimeSpan.FromSeconds(5), ct);

                var downloadUrl = $"/api/v2/transcripts/{reportId}/down
load";
                await statusStore.MarkReadyAsync(reportId, downloadUrl,
 ct);
```

```
                await notificationService.NotifyTranscriptReadyAsync(
                    request.StudentId, reportId, downloadUrl);

                logger.LogInformation(
                    "Transcript ready, notification sent: {ReportId} fo
r student {StudentId}",
                    reportId, request.StudentId);
            }
            catch (Exception ex)
            {
                logger.LogError(ex, "Transcript generation failed: {Rep
ortId}", reportId);
                await statusStore.MarkFailedAsync(reportId, ex.Message,
 CancellationToken.None);
            }
        }
    }
}
```

Add the using to the top of `TranscriptWorker.cs`:

```
using TmsApi.Application.Notifications;
```

## Step 6 Test with browser DevTools (with auto-reconnect)

Open the API in your browser (the Scalar docs page is fine). Open DevTools (F12) → Console. Run:

```
  const signalR = await import("https://cdn.jsdelivr.net/npm/@microsoft
/signalr@8.0.0/+esm");

const connection = new signalR.HubConnectionBuilder()
  .withUrl("/hubs/tms?studentId=1")
  .withAutomaticReconnect([0, 2000, 10000, 30000])
  .configureLogging(signalR.LogLevel.Information)
  .build();

connection.onreconnecting((err) =>
  console.warn("Reconnecting…", err?.message ?? ""),
);
connection.onreconnected((id) =>
  console.info("Reconnected with connection id", id),
);
connection.on("ReceiveTranscriptReady", (reportId, url) =>
  console.log(`Transcript ready: ${reportId} at ${url}`),
);
```

```
await connection.start();
console.log("Connected to TmsHub for student 1");
```

`withAutomaticReconnect([0, 2000, 10000, 30000])` tells the client to retry immediately, then after 2 s, 10 s, and 30 s before giving up. Without it, the **first** time you `dotnet run` after a code change, every connected client silently dies. With it, learners see "Reconnecting… Reconnected" log lines and then the next transcript notification arrives the experience real users expect.

Now, in another terminal, POST a transcript for student 1:

```
curl -X POST https://localhost:5001/api/v2/transcripts \
  -H "Content-Type: application/json" \
  -d '{"studentId": 1}'
```

Within ~5 seconds the browser console logs `Transcript ready: <reportId> at /api/v2/transcripts/<reportId>/download`.

Open a *second* browser tab connected with `?studentId=2`. Trigger the transcript for student 1 again. Tab 2 should see **nothing** the message went only to the `student-1` group.

**Expected result:**

- The SignalR connection establishes; logs show the connection id.
- A POST for student 1 results in **only** student-1's tab receiving `ReceiveTranscriptReady`.
- Restarting the API briefly causes the client to log "Reconnecting…" then "Reconnected" and the next transcript notification arrives normally.

## Step 7 Production: backplane and authorization (one paragraph each)

When you scale to **two API pods** behind a load balancer, the moment student 1 connects to pod A but the worker on pod B finishes their transcript, the message goes nowhere pod B doesn't know about pod A's groups. The fix is a **backplane**: every pod publishes group sends to a shared transport, every pod subscribes. Two production options:

```
// Option A: Redis backplane (self-hosted)
builder.Services.AddSignalR().AddStackExchangeRedis(
    builder.Configuration.GetConnectionString("Redis")!,
    options => options.Configuration.ChannelPrefix = "tms-signalr");

// Option B: Azure SignalR Service (managed; recommended at scale)
builder.Services.AddSignalR().AddAzureSignalR(
    builder.Configuration.GetConnectionString("AzureSignalR"));
```

You do not enable a backplane in this lab single-pod is fine for learning. You should be able to explain *when* you would (the moment the deployment goes from one replica to two).

For **authorization**, the pattern in M12 will be:

```
[Authorize]                 // require JWT for hub connection
public class TmsHub : Hub<ITmsHubClient> { ... }
```

…and `Context.UserIdentifier` replaces the `studentId` query string in the auto-join. The hub method names and group naming convention stay identical.

### Checkpoint

## Session Integration Challenge — Two students, one transcript

When both exercises are done, this scenario should "just work":

```
# Tab 1, in browser DevTools, connect as student 1:
# (paste the client snippet from Exercise 6 · Step 5 in this handout)

# Tab 2, in browser DevTools, connect as student 2.

# Now: from a terminal, generate a transcript for student 1
$key = [guid]::NewGuid()
Invoke-WebRequest -Method POST https://localhost:5001/api/v2/transcript
s \
    -Headers @{ "Idempotency-Key" = $key } \
    -Body '{"studentId": 1}' -ContentType 'application/json'

# Repeat the exact same call within a couple of seconds
Invoke-WebRequest -Method POST https://localhost:5001/api/v2/transcript
s \
    -Headers @{ "Idempotency-Key" = $key } \
    -Body '{"studentId": 1}' -ContentType 'application/json'
```

Expected behaviour:

- **First POST** returns 202 with a fresh `reportId` and status `Queued`.
- **Second POST** within seconds returns 202 with the **same** `reportId`. Worker logs show only **one** "Generating transcript …" line.
- Status endpoint progression for that `reportId`: `Queued` (instant) → `Processing` (within ~100 ms) → `Ready` (around 5 s) with `startedAt`, `completedAt`, and `downloadUrl`.

- **Tab 1** logs `Transcript ready: <reportId> at /api/v2/transcripts/<reportId>/download` exactly **once**.
- **Tab 2** logs nothing—the message went to `student-1`, not `student-2`.
- Restart the API. Both tabs log `Reconnecting…` → `Reconnected`. Repeat the POST; tab 1 receives the notification again.

That sequence is the entire long-running-async-with-realtime pattern, end to end. If any step is missing, you have a wiring gap somewhere in Session 3.

## Session 3 Checkpoint

Tick all of these before you leave the room:

- ☐ `POST /api/v2/transcripts` returns 202 immediately. The `Location` header points to the status URL. The body is the `Queued` status with `requestedAt`.
- ☐ `GET /api/v2/transcripts/{id}/status` reflects truth: `Queued` → `Processing` → `Ready`, with timestamps populated correctly and a real `downloadUrl` on `Ready`.
- ☐ A second POST with the same `Idempotency-Key` returns the **same** `reportId`. Worker logs show one—not two—"Generating transcript …" lines.
- ☐ The state machine refuses illegal transitions. (Sanity check: try forcing a transition from `Ready` back to `Queued` in a unit test and confirm it throws.)
- ☐ A SignalR client connected `?studentId=1` receives `ReceiveTranscriptReady` after a transcript for student 1 completes.
- ☐ A SignalR client connected `?studentId=2` receives **nothing** when student 1's transcript completes.
- ☐ Both clients survive a `dotnet run` restart via `withAutomaticReconnect`.
- ☐ `Clients.All` is **not** used anywhere in the codebase.
- ☐ `dotnet test` is still green.